

Northstar SellerOS

Full-Picture Handoff Briefing

Bilingual (EN / fr-CA), Canada-native autonomous real-estate operating system —
what it is, what was proven, what is still gated, and exactly what happens next.

Release verdict: Demo Pilot Deployable — suitable only for supervised Ontario testing using unmistakably fictional data, human approval gates, and mock integrations. **GATED: WAITING_FOR_OWNER_ROTATION.** Not production-ready.

Release	northstar-selleros @ 1.0.0-pilot
Freeze tag	v1.0.0-pilot (commit c63286a)
Date	2026-08-03
Prepared by	Release engineering — for colleague handoff
Classification	Internal — fictional demo data only; no real personal information



Contents

1	Executive summary	3
2	What the system is	3
2.2	The agent fleet — 20 cores, 3 wired	4
2.3	Policy enforcement — 44 rules	4
3	Architecture and technology	5
4	Release journey — how we got here	7
5	Verification evidence — what was proven	8
6	Security, privacy and compliance posture	10
7	Known limitations and residual risks	11
8	Current release state and artifacts	12
9	Operator runbook — what happens next	13
10	Go / No-Go and final verdict	14
	Appendix A Key file map	15
	Appendix B Command reference	16

1 Executive summary

Northstar SellerOS is a bilingual (English / French-Canadian), Canada-native operating system for real-estate sellers: a multi-tenant web application in which autonomous agent "cores" draft valuations, strategies, offers and transaction plans — but every externally visible action passes through policy packs, an append-only hash-chained audit log, and mandatory human approval gates. The demonstration tenant is the deliberately fictional **Harbourline Realty** brokerage; no real personal data exists anywhere in the delivery.

As of **2026-08-03** the release is **frozen** at annotated git tag `v1.0.0-pilot` (commit `c63286a`). Every gate that can be executed without owner-only platform actions is green: 253/253 automated tests (twice consecutively), 131/131 golden + 85/85 simulator evaluations, clean typecheck/lint, clean secret / fictional-data / lockfile-host / whole-delivery scans, zero production dependency vulnerabilities, migrations proven idempotent on the actual hosted TiDB database, and a 7/7 smoke battery run against the production build backed by that same database.

The one-sentence verdict. Demo Pilot Deployable — suitable only for supervised Ontario testing using unmistakably fictional data, human approval gates, and mock integrations — **GATED: WAITING_FOR_OWNER_ROTATION** (the owner must rotate two credentials in the platform console). **Not production-ready.**

Three owner-only actions remain — the agent cannot perform them and has not pretended to: (1) rotate `DATABASE_URL` and `APP_SECRET` in the platform console; (2) push the repository to the owner's git host, which turns on hosted CI automatically; (3) press the platform's publish button, then re-run the committed smoke battery against the public URL. Section 9 is the runbook for exactly these steps.

Two engineering findings are worth a colleague's attention. First, a concurrency defect class (*read-max-then-insert* sequence allocation under a unique index) was found by stress testing — not by luck — in both the workflow-event log (lost no rows in tests but carried the race) and the audit log (measurably lost 159 of 400 rows under 5-way concurrency before the fix; zero after). Second, validation against the intended hosted database revealed the provisioned schema had been created out-of-band with an empty migration journal; this was detected, drift-checked, and repaired with byte-exact baseline journaling, then re-proven. Both are documented with raw numbers in Sections 4–6.

2 What the system is

2.1 Product scope

Northstar SellerOS targets Canadian brokerages, with Ontario as the pilot jurisdiction. It is bilingual by construction (EN / fr-CA), multi-tenant (brokerage → team → agent hierarchy with

hard tenant isolation), and compliance-forward: consent records, a suppression list, CASL-style send blocking, and policy-gated autonomy levels are first-class schema objects, not afterthoughts. The pilot scope is the seller side of the transaction: property dossiers, comparables and valuations, pricing strategy, offer intake and comparison, and transaction task coordination.

Everything demonstrable today runs on the seeded fictional brokerage **Harbourline Realty** (1 tenant, 4 users at validation time). A standing consent order forbids the use of any real property owner's data; historical artifacts that contained real identifiers were quarantined out of the delivery (Section 6.2).

2.2 The agent fleet — 20 cores, 3 wired

The system is specified as **20 agent cores**. Exactly **3 are wired** end-to-end into the application; the remaining 17 exist as specified-but-unwired cores. This is stated plainly in every release document — the demo must never be represented as a 20-agent system in operation.

Table 1. Agent fleet status (verified against code, not documentation)

Core	Status	What it does in the pilot
ConversationalLead	Wired	Lead intake conversation, consent capture, dossier bootstrap
OfferExtraction	Wired	Parses incoming offers into structured terms for comparison
TransactionCoordinator	Wired	Generates and tracks the transaction task plan
17 further cores	Unwired	Specified in design; not connected to the running application — roadmap, not demo

Model calls in this release run against a **deterministic mock provider** by default; no live model or third-party integration is exercised. External integrations (e.g., messaging sends) are mocks guarded by the outbox and policy layer. Nothing in the release claims otherwise.

2.3 Policy enforcement — 44 rules

A versioned policy pack (44 rule IDs) mediates agent action: **17 rules enforced** in code, **12 partially enforced**, **15 declared** (documented, not yet executable). The flagship demonstrable rule is **CASL-03**: the seeded demo shows an SMS send to a contact with expired consent being *blocked*, with the block decision itself recorded in the audit chain. Approval gates are single-use: a concurrent double-decide race was stress-tested (10 rounds, single winner) and an approval, once consumed, cannot be replayed.

2.4 Autonomy model

Tenants carry an autonomy ceiling (e.g., A2) and per-action autonomy levels. Anything above the ceiling, anything touching an external side effect, and anything flagged by policy lands in the **approvals** queue for a human decision with an expiry. The system is therefore "autonomous" only inside a fenced yard — the fence being the point of the pilot.

3 Architecture and technology

3.1 Stack

Table 2. Technology stack and pinned baselines

Layer	Technology	Notes
Runtime	Node.js $\geq$ 22.22.0, npm $\geq$ 10	Enforced via <code>engines</code> ; react-router 8.3.0 requires $\geq$ 22.22.0 — the pin is load-bearing
Frontend	React 19 + Vite + TypeScript, Tailwind + shadcn/ui	Built to <code>dist/public/</code>
API server	Hono + tRPC 11	Bundled by esbuild to <code>dist/boot.js</code> ; externals are Node builtins + <code>mysql2</code> 3.23.2 only
Data	Drizzle ORM; MySQL 8-compatible / TiDB	Hosted target validated on TiDB v8.5.3-serverless; 32 application tables + migration journal (33 total), 60 foreign keys
Auth	Kimi OAuth, JWT HS256 sessions	<code>APP_SECRET</code> doubles as the JWT signing key — rotation invalidates all sessions (desired)
Testing	Vitest (253 tests incl. DB-backed suites), golden + simulator eval harnesses	Concurrency regression tests cover both fixed race sites
Migrations	Committed versioned migrations via <code>scripts/db-migrate.mjs</code>	<code>db:push</code> is prohibited on TiDB (truncate-prompt hazard) — enforced by CI design and docs

3.2 Runtime topology

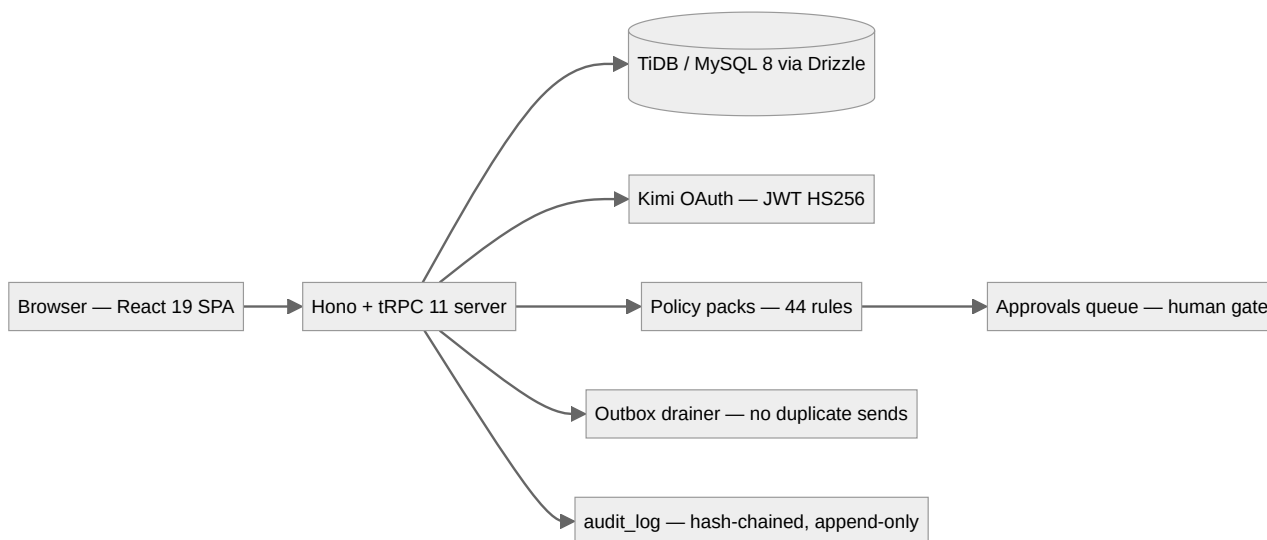


Figure 1 — request path. Every mutating route is authenticated (401 unauthenticated), tenant-scoped, policy-checked, and audit-chained; side effects drain through the outbox.

3.3 Integrity mechanisms a colleague should know

- **Hash-chained audit log.** Each `audit_log` row chains `prevHash` → `hash` over canonical fields, per tenant, with a per-tenant monotonic `seq` under the unique constraint `audit_tenant_seq(tenantId, seq)`. `verifyAuditChain()` re-validates the chain; the smoke battery runs it against live data.
- **Tenant isolation.** 27 dedicated cross-tenant escape tests (`tenantEscape.test.ts`) run inside the full suite; smoke item 8 re-verifies isolation on staging data.
- **Outbox idempotency.** Sends carry (`tenantId`, `action`, `idempotencyKey`) uniqueness; a `SIGTERM` → reboot cycle was proven to produce zero duplicate sends (outbox 2 → 2).
- **Single-use approvals.** Concurrent double-decide resolves to exactly one winner; consumed approvals cannot be replayed (6 replay-race tests).

3.4 Build and run

```
npm ci                # 1,143 packages, 0 vulnerabilities, public-registry-only
lockfile
npm run build          # vite build (frontend) + esbuild api/boot.ts → dist/boot.js
npm start              # NODE_ENV=production node dist/boot.js (PORT env)
node scripts/smoke.mjs --base <url> # committed 7-check smoke battery, any target
```

4 Release journey — how we got here

The release went through two engineering waves plus a final verification-and-freeze pass, each driven by independent audit rather than self-attestation. The through-line of the project rule: *tests and gates may not be weakened to manufacture green results; where evidence and documentation disagree, the evidence wins.*

Table 3. Release timeline (all 2026-08-03, commits on master)

Phase	Commit	What happened
Wave 1 — release-lead mandate	8e8868a	P0 credential-incident containment, data decontamination, false-green eval fix; clean release/security/db+CI/docs work; staging smoke 14/14; 12 release deliverables produced
Independent audit → STOP order	—	Clean-room audit confirmed strong core code but found 6 release blockers: private-registry lockfile URLs, banned identifiers in handoff docs, an overclaiming "production-grade" handoff, rotation marked internal instead of owner-external, CI gaps (Node 20, db:push, no evals), and no hosted CI run
Wave 2 — audit remediation	f3e19ac	Lockfile normalized to npmjs-only (707 URLs) + CI-blocking lockfile-host gate; Node baseline raised to ≥ 22.22.0; CI switched db:push → migration-proof and gained an evals step; truthful public handoff (3/20 wired, mocks, not production-ready); whole-delivery scan with no exemptions; canonical ZIP rebuilt with manifest + SHA-256
Workflow race fix	2e6b579	Stress testing exposed the read-max-then-insert race in <code>appendWorkflowEvent</code> ; fixed with retry budget 10 + jittered backoff + regression test; root cause documented (retries absorb, not remove, the race window)
Verifier pass	version 0eb6c79	Independent verifier: 17/17 gates + 14/14 smoke from a brand-new extraction of the shipped ZIP
Audit-log race found + fixed	4e1414b	Final review reproduced the same race class in <code>appendAudit</code> : 159/400 rows silently lost at 5-way concurrency. Same minimal fix pattern + regression test → 0/400 lost; full battery re-run green (253/253 ×2)
Hosted-DB validation + freeze	c63286a , tag v1.0.0- pilot	Validation on the real TiDB exposed an empty migration journal (schema created out-of-band); drift-checked, baseline-journaled, re-proven idempotent. Reusable smoke battery committed and proven 7/7 on the TiDB-backed production build. Release frozen and tagged

Two scratch-file hygiene issues were also caught by the final gate runs and corrected at freeze (a leaked stress harness `race-stress.ts` was removed from the tracked tree and the ZIP rebuilt). They are disclosed here because the handoff standard is disclosure, not cosmetics.

5 Verification evidence — what was proven

5.1 The gate wall (all executed, none waived)

Table 4. Gate results at freeze (tag v1.0.0-pilot unless noted)

Gate	Result	Evidence location
Install reproducibility (<code>npm ci</code>)	1,143 pkgs, 0 vulns	release/DEP_AUDIT.txt (6 moderate dev-only advisories disclosed)
Typecheck (<code>tsc -b</code> + <code>tsc --noEmit</code>)	clean	release/GATE_TABLE.md
Lint (<code>eslint .</code>)	0 errors, 5 pre-existing warnings	release/GATE_TABLE.md
Unit + DB-backed tests (<code>vitest run</code>)	253/253, twice consecutively (36 files)	release/GATE_TABLE.md
Evals — golden scenarios	131/131	release/GATE_TABLE.md
Evals — simulator	85/85	release/GATE_TABLE.md
Committed-secret scan	clean (298 tracked files)	release/ZERO_SECRET_SCAN.txt
Fictional-data gate (banned-identifier scan)	clean	release/FICTIONAL_DATA_SCAN.txt
Lockfile-host gate	707/707 URLs on registry.npmjs.org	CI step + <code>npm run gate:lockfile</code>
Whole-delivery scan (incl. nested ZIP)	clean — 331 files, 1 archive	release/FICTIONAL_DATA_SCAN.txt (freeze re-run)
Production dependency audit	0 high/critical	release/DEP_AUDIT.txt

5.2 Concurrency stress evidence (the audit-log race)

The defect was reproduced from a clean checkout before any fix was attempted, and the fix was accepted only on measured before/after evidence — 5-way concurrent audit appends × 80 rounds against a fresh migrated database:

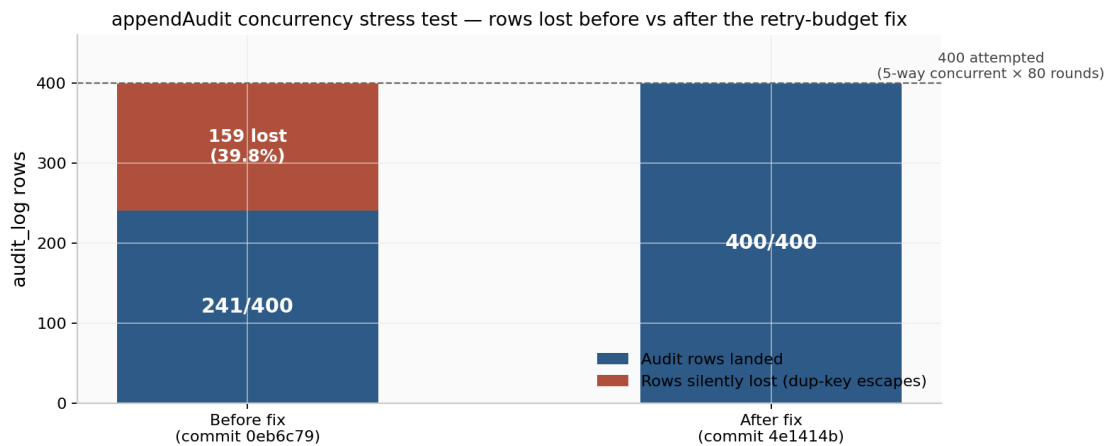


Figure 2. appendAudit stress test, before vs after commit 4e1414b. After: ok=400 dupkey_escapes=0 audit_rows=400 distinct_seq=400 min_seq=1 max_seq=400 — a complete, gapless, chain-valid sequence.

Root cause, stated honestly. Sequence numbers were allocated as *read current max, then insert max+1* under a unique index. Under concurrency, two writers read the same max; the loser's insert collides and — with the original budget of 3 retries — could exhaust and drop the row silently. The fix (budget 10, 5–20 ms jitter, tip re-read per attempt, only duplicate-key errors masked, loud exhaustion error) makes loss unobserved in 400/400 attempts but **absorbs rather than removes the race window**. The permanent cures — an atomic per-tenant/per-workflow sequence allocator (UPDATE ... last_seq+1 ... RETURNING) or transactional SELECT ... FOR UPDATE — are documented roadmap items in release/RESIDUAL_RISK.md, not completed work.

5.3 Hosted-database validation (on the real platform TiDB)

- **Finding:** the provisioned database (TiDB 8.0.11-TiDB-v8.5.3-serverless) had all 33 tables but an **empty migration journal** — the schema had been created out-of-band, so db:migrate failed with "table already exists". The first future migration would have collided in production.
- **Drift check before repair:** all 7 named unique constraints present with identical columns (including both race-guarding indexes), 60/60 foreign keys, 33/33 primary keys, spot-checked column counts — the live schema matches the committed migration, so journaling hid nothing.
- **Repair:** exactly one baseline journal row, byte-identical to what the Drizzle migrator itself writes (hash = SHA-256 of the migration file, timestamp = journal entry), verified against the migrator's source.
- **Proof after repair:** db:migrate → exit 0 no-op; the CI migration proof → migrate ×2 idempotent, an inserted audit row survived the second run, table count stable at 33, proof rows cleaned up and verified gone.

5.4 Smoke batteries

Table 5. Smoke evidence

Battery	Result	Target and scope
14-item staging battery (independent verifier)	14/14	Fresh extraction of the shipped ZIP: HTTP checks, tenant isolation, blocked-CASL demo, approval single-use, restart zero-duplicate-sends, audit-chain verify, migrate/seed idempotency, graceful shutdown
7-check committed battery (<code>scripts/smoke.mjs</code>)	7/7	Production build on Node 22.22.0 backed by the real provisioned TiDB: app shell, tRPC ping, livez, readyz (live DB SELECT 1), 401 unauthenticated, 404, OAuth 302; plus SIGTERM graceful shutdown and outbox stability (2 → 2)

Because the smoke battery is committed and takes any base URL, the owner re-runs the identical checks against the published URL after publishing — same script, same expectations (Section 9).

6 Security, privacy and compliance posture

6.1 Credential incident and rotation status

During development, the live `DATABASE_URL` and `APP_SECRET` were exposed in working transcripts. Containment was executed immediately (exposure inventory, delivery scrubbing, CI secret-scan gate), but **rotation is an owner-only platform-console action** and remains outstanding. Release status is therefore pinned to **WAITING_FOR_OWNER_ROTATION** until the owner rotates both values and redeploys. This document contains no credential values; the runbook (Section 9, step 1) tells the owner how to generate and install replacements without transmitting them.

One residual fact for the operator: the runtime working tree holds a git-ignored `.env` with the live platform values (the preview needs it). It is verified untracked and absent from the delivery ZIP; it must never be committed, zipped, or pasted.

6.2 Consent order and data decontamination

A standing consent order forbids any use of a real property owner's house, identity, documents, contacts, or transaction. Consequences executed and verified:

- All demo and test data is the fictional **Harbourline Realty** set; a banned-identifier gate (`scripts/fictional-data-gate.mjs`) blocks re-introduction.

- Historical artifacts that contained real identifiers were quarantined to `internal/` (design drafts, a raw git diff, notes) with in-place WITHHELD markers — they are excluded from the public delivery by design.
- A whole-delivery recursive scan (`scripts/delivery-scan.mjs`), including nested archives, reports zero banned identifiers across 331 files and the shipped ZIP.

6.3 Supply-chain and delivery gates

- **Lockfile-host gate** — all 707 resolved URLs point at the public npm registry; any other host fails CI.
- **Secret scan** — blocks private keys, non-placeholder secrets, and non-local database URLs across every tracked file.
- **Dependency audit** — zero high/critical in production dependencies; six moderate advisories exist in dev-only tooling and are disclosed in `release/DEP_AUDIT.txt`.

6.4 Authentication, authorization, audit

Kimi OAuth issues JWT (HS256) sessions; unauthenticated API calls receive 401 (smoke-proven). Authorization is role- and tenant-scoped with CASL-style abilities; cross-tenant escape is covered by 27 dedicated tests. Every mutating action writes a hash-chained audit row; the chain was verified on live seeded data (`{"ok":true}`).

7 Known limitations and residual risks

Table 6. Residual risks, ranked (full dispositions in `release/RESIDUAL_RISK.md`)

Severity	Item	Honest state
High (gating)	Credential rotation outstanding	Owner console action; release unusable for supervised sessions until done — <code>WAITING_FOR_OWNER_ROTATION</code>
Medium	Sequence-allocation race window absorbed, not eliminated	Retry hardening proven 0/400 losses on both sites; permanent atomic allocator on roadmap
Medium	17 of 20 agent cores unwired; model + integrations are mocks	By-design for the pilot; must never be demoed as live autonomy
Medium	Hosted CI not yet running	Workflow is turnkey and validated locally step-for-step; runs automatically once the owner pushes to their git host
Low	CI validates migrations on mysql:8; platform runs TiDB	Mitigated for this release by direct validation on the real TiDB (Section 5.3); swap the CI service image if TiDB-specific features are ever adopted

Low	No down-migrations	<code>0000_init</code> is additive-only; rollback means restoring a database snapshot (owner console), never hand-written down-migrations against the live audit chain
Low	6 moderate dev-only dependency advisories	Disclosed in release/DEP_AUDIT.txt; not reachable from production code paths

Legal/compliance honesty: nothing in this release constitutes legal certification, live CASL compliance sign-off, or validated live integrations. The compliance posture demonstrated is engineering-level (gates, audit chain, consent schema, blocked-send demo) and must be represented as such.

8 Current release state and artifacts

8.1 Release identity

Table 7. Frozen release identity (verify before trusting any copy)

Item	Value
Package	<code>northstar-selleros@1.0.0-pilot</code> (private)
Freeze tag	<code>v1.0.0-pilot</code> (annotated)
Commit	<code>c63286ad5896bd83da54b4f386461cb4437a6bce</code>
Canonical ZIP	<code>release/northstar-selleros-source.zip</code>
ZIP SHA-256	<code>fc1cabe9ba7756f61181cfbb383cc0989f904d458bdcde78e98f3d4bc51601da</code>
Platform preview version	<code>ea827eb</code> (preview snapshot; not published)
Runtime baseline	Node.js $\geq$ 22.22.0, npm $\geq$ 10
Credential status	WAITING_FOR_OWNER_ROTATION

8.2 Artifact map

Table 8. Where each piece of evidence lives

Artifact	Contents
<code>DEPLOYMENT.md</code> (repo root, in ZIP)	Final deployment documentation: rotation runbook, hosted-DB validation record, CI stand-up, publish + post-publish smoke, rollback, go/no-go
<code>release/MANIFEST.md</code> <code>SHA256SUMS.txt</code>	+ Release identity, hashes, supersession notes

<code>release/GATE_TABLE.md</code>	Exact gate outputs incl. concurrency root-cause analysis
<code>release/SMOKE_RESULTS.md</code>	14-item staging battery + deployment-target addendum (7/7)
<code>release/ROTATION_STATUS.md</code>	Incident record, containment, owner actions
<code>release/CI_STATUS.md</code>	Hosted CI = honest PENDING + workflow state + local proofs
<code>release/RESIDUAL_RISK.md</code>	Risk dispositions incl. both concurrency addenda
<code>release/DEP_AUDIT.txt</code> , <code>ZERO_SECRET_SCAN.txt</code> , <code>FICTIONAL_DATA_SCAN.txt</code>	Raw gate outputs
<code>HANDOFF.md</code> (outer)	Truthful public handoff — 3/20 cores wired, mocks, not production-ready
<code>internal/</code> (quarantine)	Contaminated historical artifacts, excluded from delivery

9 Operator runbook — what happens next

Three owner actions remain, in strict order. Everything else is done.

Step 1 — Rotate credentials (unblocks everything)

1. In the platform console, rotate the database credentials behind `DATABASE_URL`.
2. On the owner's own machine, generate a new `APP_SECRET`: `openssl rand -hex 32`. Paste it only into the platform console's secret field — never into chat, tickets, docs, or git.
3. Redeploy/restart from the console.
4. Verify: `node scripts/smoke.mjs --base <preview-url> — /api/readyz` must return `{"status": "ready"}` (proves the server reaches the database with the new URL).
5. Update `release/ROTATION_STATUS.md` from `WAITING_FOR_OWNER_ROTATION` to done.

Step 2 — Stand up hosted CI

1. Create a private repository on the owner's git host.
2. `git remote add origin <repo-url>` then `git push -u origin master --tags`.
3. The `ci` workflow runs automatically on push: 14 steps, no secrets required (all test constants; ephemeral mysql:8 service). Expected: install → lockfile-host gate → secret scan → fictional-data gate → typecheck ×2 → lint → migration proof → 253 tests → evals (131 + 85) → build → audit.

Step 3 — Publish, then re-smoke

1. Publish via the platform's manual 「发布」 action (never automated).
2. Run `node scripts/smoke.mjs --base https://<published-url>` — expect 7/7.
3. First-login check: Kimi OAuth sign-in, Harbourline Realty demo tenant loads, blocked-CASL demo behaves as documented.

Rollback

Every release state is a git tag/SHA; redeploy a previous tag from the console. The database is additive-only with no down-migrations — database rollback means restoring a pre-release snapshot (owner console responsibility).

10 Go / No-Go and final verdict

Table 9. Go / No-Go checklist

#	Item	State
1	Owner rotated DATABASE_URL + APP_SECRET	☐ WAITING_FOR_OWNER_ROTATION
2	Hosted DB validated on platform TiDB	☒ Done 2026-08-03 (journal repaired; migrate ×2 proof)
3	Hosted CI green on owner's git host	☐ Pending owner push (workflow turnkey)
4	Post-publish smoke 7/7	☐ After publish (7/7 pre-publish on TiDB-backed runtime)
5	Release frozen and tagged	☒ v1.0.0-pilot
6	Deployment documentation delivered	☒ DEPLOYMENT.md + this briefing

Final verdict. Northstar SellerOS v1.0.0-pilot is **Demo Pilot Deployable**: suitable only for supervised Ontario testing using unmistakably fictional data, human approval gates, and mock integrations — **GATED: WAITING_FOR_OWNER_ROTATION**. It is **not production-ready**: 17 of 20 agent cores are unwired, integrations are mocks, hosted CI awaits the owner's push, and credential rotation is outstanding. Every claim in this briefing is backed by an executable gate or a named artifact in the release package; where a claim could not be executed (owner-only actions), it is marked pending rather than asserted.

Appendix A Key file map

Table 10. Repository landmarks (paths relative to repo root)

Path	Purpose
<code>api/boot.ts</code> → <code>dist/boot.js</code>	Production server entry (esbuild bundle; externals: Node builtins + mysql2)
<code>api/audit.ts</code>	Hash-chained append-only audit log; retry-hardened seq allocation (4e1414b)
<code>api/store/drizzle.ts</code>	Drizzle store; workflow-event append with the same race hardening (2e6b579)
<code>db/migrations/0000_init.sql</code>	The single committed migration — schema source of truth (32 tables, 60 FKs)
<code>scripts/db-migrate.mjs</code> / <code>ci-migrate-proof.mjs</code>	Migration runner / idempotency + audit-row-survival proof
<code>scripts/smoke.mjs</code>	Reusable 7-check smoke battery for any base URL
<code>scripts/secret-scan.mjs</code> , <code>fictional-data-gate.mjs</code> , <code>lockfile-host-gate.mjs</code> , <code>delivery-scan.mjs</code>	The four content/supply-chain gates
<code>.github/workflows/ci.yml</code>	Turnkey hosted CI (14 steps; runs on push to main/master/final-build/release-*)
<code>evals/</code>	Golden scenarios (131) + simulator (85) harnesses
<code>.env.example</code>	Complete environment template — the only env file in the delivery

Appendix B Command reference

```
# reproduce the environment
git checkout v1.0.0-pilot && npm ci                                # Node ≥ 22.22.0

# full local gate battery
npm run check && npm run lint && npm test                          # typecheck, lint, 253 tests
npm run evals                                                    # 131 golden + 85 simulator
node scripts/secret-scan.mjs                                     # committed-secret gate
node scripts/fictional-data-gate.mjs                            # banned-identifier gate
npm run gate:lockfile                                           # 707 npmjs-only URLs

# database (never db:push against TiDB)
npm run db:migrate                                              # committed migrations, idempotent
node scripts/ci-migrate-proof.mjs                               # migrate x2 + audit-row survival proof

# build, run, smoke
npm run build && npm start                                         # dist/boot.js on $PORT
node scripts/smoke.mjs --base <url>                             # 7/7 expected

# verify a delivered ZIP
sha256sum northstar-selleros-source.zip                        # must equal fc1cabe9...60ea1
(SHA256SUMS.txt)
```

End of briefing. Questions of fact should be checked against the artifacts in Table 8 — if this document and the evidence ever disagree, the evidence is correct and this document should be fixed.